

menler

Your turning point in the AI era.

AI APTITUDE QUESTION BANK

LLM

Fundamentals

About This Question Bank

This paper covers five core competency areas in agentic AI systems and workflow automation. Each set tests practitioner-level knowledge: not definitions, but the ability to diagnose architectural failures, select the correct design pattern, and evaluate tradeoffs in real-world agentic deployments.

5

Sets

15

Questions per Set

75

Questions Total

SET GUIDE

SET 1

Agent Architecture & Planning Patterns

Agent types, planning patterns, multi-agent orchestration, tool count, memory, checkpoint, clarification

SET 2

Workflow Automation, Triggers & Error Handling

Trigger design, batch vs event, DLQ, idempotency, error branching, schema evolution, SLA patterns

SET 3

Tool Design, MCP & API Integration

Tool grounding, error taxonomy, caching, rate limiting, security, inter-agent contracts, prompt injection

SET 4

Memory Systems, Context & Multi-Session Agents

Episodic / semantic / procedural memory, context compression, multi-tenant isolation, handoff summaries

SET 5

Safety, Governance & Production Readiness

Audit logging, canary deployment, scope enforcement, graceful degradation, proxy discrimination, governance

Each question has one correct answer. Select the best answer from the options provided.

Correct answers and explanations follow each question to reinforce the concept.

SET

1

Transformers, Tokens & Inference

Set 1 of 5 · 15 Questions · LLM Fundamentals Question Bank

Q1. A production LLM application starts returning truncated responses on inputs that previously worked fine. No code changed. Which is the most likely cause?

- A. The model's temperature was automatically increased by the provider.
- B. The input grew longer over time (e.g. conversation history), consuming more of the context window and leaving less room for the output.**
- C. The embedding model changed, causing token count mismatches.
- D. The API's rate limiter is cutting responses short.

Correct Answer: B

Explanation

Context windows are shared between input and output. As conversation history accumulates, the input consumes more tokens, shrinking the space available for the completion. This is one of the most common silent failure modes in stateful LLM applications.

Q2. You are choosing between a 7B and a 70B parameter model for a production summarisation task. The 7B is 4× cheaper and 3× faster. When does the 70B model justify the cost?

- A. Always — larger models are always better.
- B. When the task requires nuanced reasoning, handling edge cases, or domain-specific judgment that the 7B model fails on in evaluation.**
- C. When the input documents are longer than 1,000 tokens.
- D. When you need to run more than 100 requests per minute.

Correct Answer: B

Explanation

Model size should be chosen by task complexity, not by default. For many straightforward summarisation tasks, a well-prompted 7B matches or approaches 70B quality. The larger model is justified when evaluation data shows the smaller model fails on the actual task difficulty.

Q3. An LLM is generating a list of 10 items but always stops at item 7. Temperature is 0, max_tokens is 2000. What should you investigate first?

- A. The stop sequence configuration — a stop token may be triggering at item 7.**
- B. The model's context window is too small.
- C. The system prompt is too long.
- D. The model has a built-in list length limit.

Correct Answer: A

Explanation

Stop sequences instruct the model to halt generation when a specific token or string appears. If a stop sequence accidentally matches content in the list (e.g. a newline or a number pattern), the model will stop early regardless of max_tokens. This is the first thing to check.

Q4. What is the practical consequence of a model having a smaller vocabulary (fewer unique tokens)?

- A. The model generates faster because it has fewer tokens to choose from.
- B. Common words in underrepresented languages get split into many subword tokens, increasing token count and reducing effective context for those inputs.**
- C. The model produces shorter outputs on average.
- D. The model cannot process inputs longer than its vocabulary size.

Correct Answer: B

Explanation

Tokenisers trained predominantly on English split non-English text into more subword fragments, consuming more tokens per word. A sentence in Hindi or Tamil may use 3–5× as many tokens as the same sentence in English, effectively shrinking the usable context window for those languages.

Q5. You need an LLM to process a 200-page legal document in a single call. Your model has a 128k token context window. What is the most important thing to verify before relying on this?

- A. That the model's temperature is set to 0 for consistent outputs.
- B. That the model performs well on content placed in the middle of very long contexts — not just at the start and end.**
- C. That the document is converted to plain text before submission.
- D. That you are using the largest available model for the task.

Correct Answer: B

Explanation

The 'lost in the middle' problem is well-documented: LLMs tend to attend more strongly to the beginning and end of long contexts. Verified performance across the full context length — especially the middle — is critical before trusting long-context processing for important documents.

Q6. An LLM correctly answers a question in testing but gives a different (also plausible) answer in production on the same input. Temperature is 0.7 in both environments. What explains this?

- A. The model is non-deterministic at temperatures above 0; different valid completions are sampled each time.
- B. The production model is a different version than the test model.
- C. The system prompt changed between environments.
- D. All of the above are plausible explanations that should be investigated.**

Correct Answer: D

Explanation

Three separate issues could cause this: (1) temperature > 0 introduces sampling variance, (2) model versions differ between environments, (3) system prompts differ. All three are common production bugs and should be checked systematically before drawing conclusions.

Q7. Which inference parameter most directly controls the trade-off between response creativity and factual reliability?

- A. max_tokens — limits how long the model can respond.
- B. top_p — controls the cumulative probability mass of tokens considered at each step.
- C. temperature — scales the probability distribution, making outputs more uniform (low) or more varied (high).**
- D. frequency_penalty — reduces repetition of already-used tokens.

Correct Answer: C

Explanation

Temperature directly scales the logit distribution. Low temperature (near 0) makes the model more deterministic and conservative, favouring high-probability tokens. High temperature increases variance and creativity but also increases the chance of factual errors or incoherence.

Q8. A model with 8k context is being used for a chatbot. After 20 turns, responses become incoherent. What is the correct architectural fix?

- A. Increase the model's temperature to help it 'recover' from long contexts.
- B. Implement a sliding window or conversation summarisation strategy to keep context within limits while preserving relevant history.**
- C. Switch to a model with a 1M token context window.
- D. Clear the conversation history every 10 turns.

Correct Answer: B

Explanation

A sliding window (discarding oldest turns) or a summarisation approach (compressing history into a compact summary) keeps the context within bounds while preserving conversational continuity. Simply clearing history breaks coherence; upgrading context windows is often unnecessary and expensive.

Q9. What does 'autoregressive generation' mean in the context of LLMs, and what is its key practical implication?

- A. The model generates all tokens in parallel, making it very fast.
- B. The model generates one token at a time, with each token conditioned on all previous tokens — meaning output length directly determines latency.**
- C. The model automatically selects the best response from multiple candidates.
- D. The model uses a feedback loop to improve outputs over multiple passes.

Correct Answer: B

Explanation

LLMs generate tokens sequentially: each token is produced based on the preceding context. This means a 1000-token response takes approximately 10× longer to generate than a 100-token response. Latency scales with output length, making output length control a key performance lever.

Q10. You observe that an LLM performs well on single-hop questions ('What is the capital of France?') but fails on multi-hop questions ('Which country's capital is also the EU's tallest building's city?'). What capability is being tested?

- A. Retrieval speed — the model cannot access its knowledge fast enough.
- B. Multi-step reasoning — the model must chain multiple facts before producing an answer.**
- C. Context length — multi-hop questions are longer.
- D. Tokenisation — compound questions are tokenised differently.

Correct Answer: B

Explanation

Multi-hop questions require the model to retrieve and chain multiple intermediate facts in sequence. This tests reasoning depth, not just knowledge retrieval. Chain-of-thought prompting significantly improves performance on multi-hop tasks by externalising the reasoning steps.

Q11. A company needs consistent JSON output from an LLM for downstream processing. The model sometimes outputs valid JSON, sometimes adds preamble text. What is the most robust fix?

- A. Set temperature to 0 and hope for consistency.
- B. Use structured output / JSON mode (if available) or add strict output format instructions with few-shot examples of correct JSON, and validate/parse the output programmatically.**
- C. Post-process the output with a regex to extract JSON.
- D. Fine-tune the model on JSON examples.

Correct Answer: B

Explanation

Structured output mode forces the model to conform to a JSON schema at the generation level. When unavailable, strict prompting with few-shot examples combined with programmatic validation provides the next best reliability. Regex is brittle; fine-tuning is overkill for a formatting problem.

Q12. What is 'perplexity' in the context of language models, and why does it matter for practitioners?

- A. A measure of how confused users feel after reading model outputs.
- B. A measure of how well a model predicts a held-out text sample — lower perplexity means the model assigns higher probability to the actual next tokens.**
- C. The number of parameters a model has divided by its training data size.
- D. A metric for how often the model produces hallucinations.

Correct Answer: B

Explanation

Perplexity measures how surprised a model is by a text sample. Lower perplexity indicates a better fit to the distribution. It's used to compare models on the same held-out data, but it doesn't directly measure factual accuracy or task performance — a model can have low perplexity and still hallucinate.

Q13. An application sends 500 tokens of system prompt + 200 tokens of user message to a model with an 8k context window. The model is asked to generate up to 2000 tokens. How many tokens remain unused?

- A. 5,300 tokens.
- B. 7,300 tokens.
- C. 6,800 tokens.
- D. 5,800 tokens.

Correct Answer: A

Explanation

$8,000 - 500 \text{ (system)} - 200 \text{ (user)} - 2,000 \text{ (max output)} = 5,300 \text{ tokens of unused capacity.}$ Understanding this arithmetic is essential for optimising context usage, especially in multi-turn applications where history accumulates.

Q14. Which statement about model 'hallucinations' is most accurate for a practitioner designing a production system?

- A. Hallucinations only occur when the model is asked about topics outside its training data.
- B. Hallucinations can occur even on topics well within training data and cannot be fully eliminated; systems must be designed to detect and mitigate them.**
- C. Setting temperature to 0 eliminates hallucinations.
- D. Larger models do not hallucinate.

Correct Answer: B

Explanation

Hallucinations are an intrinsic property of language models — they can confabulate on any topic, including well-known facts. Temperature = 0 reduces variance but not hallucination risk. Larger models hallucinate less often but not never. Robust systems use retrieval grounding, output validation, and human review for high-stakes claims.

Q15. You need to run an LLM on a device with 16GB of RAM. A model has 7B parameters in float32 precision. Approximately how much RAM does it require?

- A. 7 GB.
- B. 14 GB.
- C. 28 GB.**
- D. 56 GB.

Correct Answer: C

Explanation

Float32 uses 4 bytes per parameter. $7B \times 4 \text{ bytes} = 28 \text{ GB}$ — more than 16 GB RAM. To run a 7B model on 16 GB, you would need to quantise it to 4-bit (approx. 3.5 GB) or 8-bit (approx. 7 GB). Understanding this arithmetic is essential for on-device and edge deployment planning.

SET

2

Embeddings, Attention & Architecture

Set 2 of 5 · 15 Questions · LLM Fundamentals Question Bank

Q1. Two documents have an embedding cosine similarity of 0.97. A user expects this to mean they are nearly identical in meaning. When would this assumption fail?

- A. When the documents are in different languages.
- B. When the embedding model was trained on a different domain than the documents.
- C. When the documents share surface-level vocabulary but express opposite sentiments or contradictory claims.**
- D. When the documents exceed 512 tokens.

Correct Answer: C

Explanation

High cosine similarity indicates similar token distributions, not identical meaning. Documents that use the same words to argue opposite sides of a debate can have very high similarity scores. Embedding models capture statistical co-occurrence, not logical entailment or factual agreement.

Q2. You are building a semantic search system. Users search across technical documentation in English, but some documents contain code, tables, and structured data. Which embedding strategy is most appropriate?

- A. Embed entire documents as single vectors using a general-purpose embedding model.
- B. Use a code-aware or mixed-content embedding model and chunk documents by semantic section before embedding.**
- C. Strip all code and tables before embedding to reduce noise.
- D. Use keyword search only for technical content and embeddings only for prose.

Correct Answer: B

Explanation

General-purpose embedding models handle prose but may poorly represent code semantics. Chunking by semantic section ensures retrieval returns targeted, relevant content rather than entire documents. Code-aware models (or purpose-built technical embedding models) significantly improve retrieval quality on mixed content.

Q3. The self-attention mechanism in a transformer assigns attention weights to tokens in the input. What does a high attention weight between two tokens indicate?

- A. The two tokens are adjacent in the input sequence.
- B. The model has determined these tokens are strongly relevant to each other for predicting the current output.**
- C. The two tokens have similar embeddings.
- D. The tokens were frequently co-occurring in the training data.

Correct Answer: B

Explanation

Attention weights represent learned relevance relationships. A high weight between token A and token B means the model is using token B's information heavily when contextualising token A. This dynamic, content-based weighting is what allows transformers to capture long-range dependencies.

Q4. An embedding model has a maximum input length of 512 tokens. A user's documents average 2,000 tokens. What is the most appropriate handling strategy?

- A. Truncate all documents to 512 tokens before embedding.
- B. Chunk documents into overlapping segments of ≤ 512 tokens, embed each chunk, and store them separately for retrieval.**
- C. Summarise all documents to under 512 tokens before embedding.
- D. Use a larger embedding model with a longer context window.

Correct Answer: B

Explanation

Chunking with overlap preserves context across boundaries and allows precise retrieval of the relevant section. Truncation loses most of the document. Summarisation loses specifics needed for precise retrieval. Upgrading to a longer-context embedding model is an option but chunking remains best practice for retrieval granularity.

Q5. What is the role of positional encodings in a transformer architecture?

- A. They compress the input sequence to reduce memory usage.
- B. They inject information about each token's position in the sequence, since the attention mechanism itself is order-invariant.**
- C. They weight tokens by how frequently they appeared in training data.
- D. They normalise token embeddings to prevent gradient explosion.

Correct Answer: B

Explanation

Self-attention computes relationships between all token pairs simultaneously and has no inherent sense of order. Positional encodings (learned or sinusoidal) add position information to token embeddings so the model knows token 5 comes before token 6. Without them, the model would treat the sequence as a bag of tokens.

Q6. You query a vector database for the top-5 most similar documents to a user's question, but the retrieved documents are consistently off-topic. The embeddings and similarity scores look correct. What should you investigate?

- A. The vector database's indexing algorithm may be approximate and introducing error.
- B. The query and documents may have been embedded with different models, creating an incompatible embedding space.**
- C. The similarity threshold is set too low.
- D. The documents need to be re-chunked at a smaller granularity.

Correct Answer: B

Explanation

Embedding spaces are model-specific. If queries are embedded with Model A and documents with Model B, cosine similarity scores are meaningless — the vectors live in different spaces. This is one of the most common and hard-to-spot RAG bugs. Always use the same embedding model for both indexing and querying.

Q7. What distinguishes a decoder-only model (like GPT) from an encoder-decoder model (like T5) in practical terms?

- A. Decoder-only models cannot be fine-tuned; encoder-decoder models can.
- B. Decoder-only models are optimised for text generation; encoder-decoder models are optimised for sequence-to-sequence tasks like translation and summarisation.**
- C. Decoder-only models are smaller; encoder-decoder models are larger.
- D. Decoder-only models use self-attention; encoder-decoder models use cross-attention only.

Correct Answer: B

Explanation

Decoder-only models (GPT family) generate text autoregressively and excel at open-ended generation. Encoder-decoder models (T5, BART) explicitly encode an input and decode an output, making them well-suited for tasks with a clear input-output structure like translation, summarisation, or question answering.

Q8. Why does increasing the number of attention heads in a transformer generally improve model quality, up to a point?

- A. More heads mean the model can compute attention faster in parallel.
- B. Each head can specialise in capturing different types of relationships (syntax, coreference, semantics), giving the model richer representations.**
- C. More heads increase the model's context window.
- D. More heads reduce the model's memory usage during inference.

Correct Answer: B

Explanation

Multi-head attention allows the model to attend to the input from multiple representational subspaces simultaneously. Different heads empirically learn to capture different linguistic phenomena. Beyond a certain point, heads become redundant and the benefit plateaus while compute cost grows.

Q9. An engineer uses an embedding model to cluster customer feedback. The clusters are dominated by sentence length rather than topic. What is the most likely cause?

- A. The embedding model is weighting longer sentences more heavily by design.
- B. The embedding model is using mean-pooling of token embeddings, causing longer texts to have different magnitude distributions that influence cosine similarity.**
- C. The clustering algorithm is not suitable for high-dimensional embedding spaces.
- D. Customer feedback is too noisy for embedding-based clustering.

Correct Answer: B

Explanation

When embedding models use mean-pooling, the resulting vector is influenced by the number of tokens averaged. Texts of very different lengths can end up in different regions of the embedding space based on length artifacts rather than meaning. Using a model with a dedicated [CLS] token or normalised pooling reduces this effect.

Q10. What is the key practical difference between 'zero-shot' and 'few-shot' prompting for a task like entity extraction?

- A. Zero-shot is only possible with very large models; few-shot works with any model.
- B. Zero-shot relies on the model's pre-trained knowledge of the task format; few-shot provides examples in the prompt that demonstrate the expected input-output pattern, significantly improving reliability.**
- C. Few-shot requires fine-tuning; zero-shot does not.
- D. Zero-shot is faster because the prompt is shorter.

Correct Answer: B

Explanation

Few-shot examples anchor the model to the exact format and level of detail expected. For structured tasks like entity extraction, few-shot prompting dramatically reduces format inconsistencies and improves precision — especially on smaller or less instruction-tuned models.

Q11. A model's MMLU benchmark score is 85%. A team uses this to justify deploying it for their internal legal document classification task. What is the risk of this reasoning?

- A. MMLU is measured on a different scale than production accuracy.
- B. MMLU tests breadth of general knowledge across many subjects; it does not predict performance on a specific, domain-specific task like internal legal classification.**
- C. MMLU scores above 80% indicate overfitting to the benchmark.
- D. Benchmark scores are only valid for English-language deployments.

Correct Answer: B

Explanation

Benchmark scores measure performance on standardised test sets, not on your specific task and data distribution. A model can score 85% on MMLU while performing poorly on legal document classification due to domain-specific vocabulary, formatting, or task structure. Always evaluate on a representative sample of your own task.

Q12. What happens when you set top_p to 0.1 during LLM inference?

- A. The model only considers the top 10% of tokens by raw count.
- B. The model restricts token sampling to the smallest set of tokens whose cumulative probability reaches 10%, making outputs very conservative and predictable.**
- C. The model skips 10% of the input tokens during processing.
- D. The model generates 10% fewer tokens than the max_tokens setting.

Correct Answer: B

Explanation

Top-p (nucleus) sampling builds a set of candidate tokens by cumulative probability. At top_p = 0.1, only the very highest-probability tokens (a tiny set) are eligible for sampling. This produces highly conservative, predictable outputs — useful for factual tasks, but it can make creative outputs repetitive.

Q13. A retrieval system returns the correct document 80% of the time, but the LLM still gives wrong answers 40% of the time. What is the most likely explanation?

- A. The LLM is ignoring the retrieved document and answering from parametric memory.
- B. The retrieved document is correct but the LLM fails to extract the specific information needed, either due to poor grounding instructions or the answer being in a hard-to-parse format.**
- C. The embedding model is not powerful enough.
- D. The LLM has a lower accuracy limit that cannot be exceeded with RAG.

Correct Answer: B

Explanation

Retrieval and generation are separate failure points. The document arriving in context does not guarantee the model will correctly extract and use the relevant information. Poor grounding instructions, dense tables, or deeply nested answers in documents all cause generation failures even with correct retrieval.

Q14. Which of the following best describes what a language model learns during pre-training?

- A. A lookup table of facts and their correct answers.
- B. A statistical model of language: the probability distribution of tokens given preceding context, trained on a massive corpus.**
- C. A set of rules for grammar and syntax.
- D. A classification function that maps inputs to predefined output categories.

Correct Answer: B

Explanation

Pre-training optimises the model to predict the next token given prior context (or masked tokens in masked LM). This simple objective, applied at massive scale, causes the model to implicitly learn grammar, facts, reasoning patterns, and world knowledge — but as compressed statistical patterns, not explicit rules or a knowledge base.

Q15. An LLM outputs the same response regardless of changes to the user's question. Temperature is 0.7. What is the most likely cause?

- A. The model's context window is too small to process the question.
- B. The system prompt is so long and prescriptive that it overrides the user's question, causing the model to always follow the same fixed template.**
- C. The model has been fine-tuned to produce a fixed response.
- D. The API is returning a cached response.

Correct Answer: B

Explanation

Overly rigid system prompts can dominate the model's attention, effectively drowning out the user's input. This is a common bug in heavily templated applications. The fix is to simplify the system prompt, explicitly instruct the model to respond to the user's specific question, and verify that user input is appearing in the context as expected.

SET

3

Fine-tuning, RLHF & Model Behaviour

Set 3 of 5 · 15 Questions · LLM Fundamentals Question Bank

Q1. A team fine-tunes an LLM on 500 customer support conversations. In production, the fine-tuned model is less helpful on edge cases than the base model. What most likely happened?

- A. The fine-tuning dataset was too small and the model overfit to the training distribution, losing general-purpose capability.**
- B. Fine-tuning always reduces model capability — it should not be used for customer support.
- C. The learning rate was too low during fine-tuning.
- D. The model's context window was reduced by fine-tuning.

Correct Answer: A

Explanation

Fine-tuning on a narrow dataset causes 'catastrophic forgetting' of broader capabilities. 500 conversations is small; the model learns the training distribution well but loses generalisation. Better approaches: use a much larger, more diverse dataset, apply LoRA/PEFT to preserve base capabilities, or use RAG instead of fine-tuning.

Q2. What is the primary purpose of RLHF (Reinforcement Learning from Human Feedback) in LLM training?

- A. To make the model faster at inference time.
- B. To align the model's outputs with human preferences — making responses more helpful, harmless, and honest — beyond what supervised pre-training achieves.**
- C. To reduce the model's hallucination rate to zero.
- D. To fine-tune the model on domain-specific data.

Correct Answer: B

Explanation

RLHF trains a reward model on human preference comparisons, then uses reinforcement learning to nudge the LLM toward outputs humans prefer. It improves instruction-following, helpfulness, and safety — but does not eliminate hallucinations and introduces its own failure modes like reward hacking.

Q3. When should you choose LoRA (Low-Rank Adaptation) over full fine-tuning?

- A. When you have more than 1 million training examples.
- B. When you have limited GPU memory or compute budget, or want to preserve the base model's general capabilities while adapting it to a specific style or task.**
- C. When the task requires the model to learn entirely new knowledge.
- D. When you need to update the model's tokeniser.

Correct Answer: B

Explanation

LoRA adds small trainable rank-decomposition matrices to existing weights, training <1% of parameters compared to full fine-tuning. This dramatically reduces memory and compute requirements while preserving most base model capabilities. It's the standard choice for resource-constrained adaptation or when catastrophic forgetting is a concern.

Q4. A fine-tuned model performs well on the held-out test set but poorly in production. The test set was sampled from the same conversations used for training. What is the fundamental evaluation error?

- A. The model was fine-tuned for too many epochs.
- B. The test set is not a representative sample of production inputs — it reflects training distribution, not real-world distribution, making evaluation results misleading.**
- C. The model's max_tokens should have been adjusted during evaluation.
- D. The evaluation metric (e.g. BLEU) doesn't apply to this task.

Correct Answer: B

Explanation

A test set sampled from the same source as training data measures in-distribution performance. Production inputs are different — different phrasings, edge cases, and topics. A proper evaluation requires a held-out set that reflects actual production distribution, not a subset of training data.

Q5. Which of the following is a direct risk of instruction fine-tuning on synthetic data generated by a more capable model?

- A. The fine-tuned model becomes larger than the teacher model.
- B. The fine-tuned model may learn to imitate the superficial style of the teacher (confident tone, fluency) without developing the underlying reasoning capability.**
- C. The fine-tuned model's tokeniser becomes incompatible with the original.
- D. Synthetic data always introduces security vulnerabilities.

Correct Answer: B

Explanation

Models fine-tuned on strong teacher output can learn to produce fluent, confident-sounding responses (mimicking style) without acquiring the teacher's reasoning depth. This is sometimes called 'false fluency' — the model sounds capable but fails on novel reasoning tasks that require genuine understanding.

Q6. You are evaluating whether to fine-tune or use RAG for grounding a model in your company's internal knowledge base of 10,000 documents. Which factor most strongly favours RAG?

- A. You have a large GPU cluster available.
- B. The knowledge base is updated frequently — new documents are added weekly.**
- C. You want the model to adopt a specific response style.
- D. You need the model to answer in a language other than English.

Correct Answer: B

Explanation

Fine-tuning bakes knowledge into weights at a point in time. A frequently updated knowledge base would require constant re-fine-tuning. RAG reads from a live document store, making updates immediate. For dynamic knowledge, RAG is significantly more practical than fine-tuning.

Q7. After RLHF training, a model becomes excessively cautious — refusing to answer many safe questions and adding unnecessary caveats. What is this failure mode called, and what causes it?

- A. Prompt injection — external inputs overrode the safety training.
- B. Over-refusal / reward over-optimisation — the reward model over-penalised anything that could be seen as harmful, causing the policy model to err too far toward refusal.**
- C. Catastrophic forgetting — the base model's knowledge was lost during RLHF.
- D. Hallucination — the model is generating false safety concerns.

Correct Answer: B

Explanation

When the reward model penalises potentially harmful content strongly, the RL optimisation can produce a policy that over-generalises safety to safe content. This is 'Goodhart's Law' applied to RLHF: optimising the measurable reward (avoiding flagged content) deviates from the true goal (being helpfully safe).

Q8. A team wants to make an LLM respond in a specific persona (tone, style, formality) without changing its factual knowledge. What is the most cost-effective approach?

- A. Full fine-tuning on persona examples.
- B. A detailed system prompt defining the persona, validated with few-shot examples of the desired style.**
- C. Training a separate model dedicated to persona formatting.
- D. Using a different base model with the desired personality.

Correct Answer: B

Explanation

Persona and style adaptation is well within what a detailed system prompt can achieve for most LLMs. Full fine-tuning is expensive and unnecessary for stylistic changes that don't require new knowledge. A system prompt with few-shot style examples is the correct first approach.

Q9. What is 'catastrophic forgetting' in the context of LLM fine-tuning, and which technique most directly addresses it?

- A. The model forgets its system prompt during long conversations; addressed by prompt caching.
- B. The model loses pre-trained general capabilities after fine-tuning on a narrow dataset; addressed by parameter-efficient methods like LoRA or by mixing general data into fine-tuning.**
- C. The model forgets user preferences between sessions; addressed by persistent memory.
- D. The model loses performance on long inputs after fine-tuning; addressed by increasing context window.

Correct Answer: B

Explanation

Catastrophic forgetting occurs when gradient updates for new task examples overwrite weights that encode pre-trained general knowledge. LoRA mitigates this by keeping most weights frozen. Mixing general-purpose examples into fine-tuning data also preserves broad capability while adapting to the new task.

Q10. A fine-tuned model is deployed and users start asking it questions outside its fine-tuning domain. It confidently produces wrong answers instead of saying 'I don't know.' What fine-tuning data characteristic caused this?

- A. The fine-tuning data had high lexical diversity.
- B. The fine-tuning data lacked examples of uncertainty expression or out-of-scope refusals, so the model learned to always produce confident answers.**
- C. The model was fine-tuned with too low a learning rate.
- D. The fine-tuning data was too long.

Correct Answer: B

Explanation

Models learn behaviours from training examples. If the fine-tuning set never showed the model saying 'I don't know' or declining to answer, the model learns that confident answers are always appropriate. Including examples of appropriate uncertainty and out-of-scope handling is essential in fine-tuning data design.

Q11. What is the difference between instruction fine-tuning and task-specific fine-tuning?

- A. Instruction fine-tuning trains the model on a single task; task-specific fine-tuning trains on many tasks.
- B. Instruction fine-tuning trains the model to follow natural language instructions across diverse tasks; task-specific fine-tuning optimises for one particular task using labelled examples.**
- C. Instruction fine-tuning requires RLHF; task-specific fine-tuning does not.
- D. They are the same — both terms describe supervised fine-tuning.

Correct Answer: B

Explanation

Instruction fine-tuning (e.g. FLAN, InstructGPT) trains on thousands of diverse instruction-response pairs, teaching the model to generalise to new instructions. Task-specific fine-tuning optimises for one task (e.g. sentiment classification) with labelled data for that task only. Instruction-tuned models are more flexible but may be less optimal for a single narrow task.

Q12. A model fine-tuned on medical QA data achieves 92% accuracy on in-domain questions. Before deploying in a clinical setting, what is the most important additional evaluation?

- A. Test inference latency to ensure it responds within 2 seconds.
- B. Evaluate calibration — whether the model's confidence scores align with its actual accuracy — and test on edge cases and adversarial medical inputs.**
- C. Run a user satisfaction survey with general users.
- D. Verify that the model passes MMLU medical benchmarks.

Correct Answer: B

Explanation

Clinical settings require not just accuracy but calibrated confidence. A model that is 92% accurate but 99% confident on wrong answers is dangerous. Calibration testing, edge case coverage, and adversarial testing are required before any clinical deployment, far beyond headline accuracy on a held-out test set.

Q13. Why is it important to use a diverse, balanced dataset when fine-tuning an LLM for a classification task?

- A. Diverse data makes fine-tuning faster.
- B. Imbalanced datasets cause the model to be biased toward the majority class, producing a model that appears accurate overall but fails on underrepresented categories.**
- C. Diverse data reduces the model's context window requirements.
- D. Balanced datasets are required by LLM licensing agreements.

Correct Answer: B

Explanation

A model trained on 90% class A / 10% class B examples learns to predict class A most of the time. This produces misleadingly high aggregate accuracy while performing poorly on class B. Class-balanced datasets, upsampling, or weighted loss functions are standard mitigations.

Q14. A customer asks: 'Should we fine-tune or use prompting to adapt our LLM?' What is the most useful framing of this decision?

- A. Always fine-tune — it always produces better results.
- B. Always use prompting — fine-tuning is too expensive.
- C. Exhaust prompting and RAG first; fine-tune only when they demonstrably fail on your task, since fine-tuning adds cost, maintenance, and evaluation overhead.**
- D. Fine-tune if you have more than 1,000 examples; otherwise use prompting.

Correct Answer: C

Explanation

Fine-tuning is a last resort, not a first step. It requires a high-quality dataset, compute, ongoing retraining as data changes, and new evaluation pipelines. Prompting and RAG are faster to iterate and often sufficient. Fine-tune when evaluation proves the gap cannot be closed by better prompting.

Q15. What is 'reward hacking' in the context of RLHF-trained models?

- A. A security vulnerability that allows users to manipulate model outputs.
- B. The model learns to optimise the reward model's score by exploiting its weaknesses rather than genuinely improving response quality.**
- C. A training bug that causes the reward signal to be inverted.
- D. When users give fake positive feedback to improve model outputs.

Correct Answer: B

Explanation

The reward model is an imperfect proxy for human preferences. During RL training, the policy model can find strategies that score highly on the reward model but produce responses humans actually rate as worse — verbose, flattering, or superficially structured. This is a core challenge in RLHF alignment research.

SET

4

Hallucinations, Evaluation & Reliability

Set 4 of 5 · 15 Questions · LLM Fundamentals Question Bank

Q1. An LLM is asked to cite sources for its answers. It produces plausible-looking citations (author, journal, year) that do not exist. What is this called, and what is the correct mitigation?

- A. Plagiarism — mitigate by checking against existing databases.
- B. Citation hallucination — mitigate by only allowing the model to cite from a verified, retrieved set of sources rather than generating citations from parametric memory.**
- C. Formatting error — mitigate by adjusting the output format in the system prompt.
- D. Context overflow — mitigate by reducing the input length.

Correct Answer: B

Explanation

Citation hallucination is common because the model learns the format of citations but invents the content. The only reliable fix is constraining the model to cite only from sources explicitly provided in context (RAG). Grounding citations in retrieved documents makes fabrication structurally impossible.

Q2. You are building an evaluation pipeline for a customer support LLM. Which combination of metrics gives the most complete picture of output quality?

- A. BLEU score and response length.
- B. Factual accuracy (against a verified knowledge base), task completion rate, and tone/helpfulness scored by an LLM judge calibrated to human raters.**
- C. User satisfaction and latency.
- D. Perplexity and vocabulary diversity.

Correct Answer: B

Explanation

BLEU measures string overlap, not quality. Latency and satisfaction measure speed and perception. A well-designed evaluation combines factual accuracy (correctness), task completion (utility), and human-aligned quality scoring. LLM-as-judge with human calibration scales efficiently while tracking dimensions that matter.

Q3. A model's outputs look confident and fluent but are factually wrong 20% of the time. A user believes the responses because they sound authoritative. What design intervention most reduces real-world harm?

- A. Add a disclaimer at the end of every response.
- B. Include calibrated uncertainty markers in the prompt instructions (e.g. 'Express uncertainty when you are not sure'), and add a fact-checking layer before displaying high-stakes claims.**
- C. Reduce the model's temperature to make it less creative.
- D. Switch to a larger model with lower hallucination rates.

Correct Answer: B

Explanation

Blanket disclaimers are ignored. The root issue is that confident tone misleads users. Instructing the model to express uncertainty is a partial fix; a fact-checking or retrieval-grounding layer before displaying claims is the structural solution. Both together address the problem at prompt level and system level.

Q4. You run the same evaluation prompt 10 times on a model (temperature 0.7) and get 10 different quality scores from an LLM judge. What is the most statistically correct way to report performance?

- A. Report the best score, as it represents the model's ceiling capability.
- B. Report the mean score with standard deviation, and run enough evaluations to achieve a stable estimate.**
- C. Report the median score to avoid outliers.
- D. Discard results and use a deterministic evaluation method only.

Correct Answer: B

Explanation

Non-zero temperature produces variance in both model outputs and LLM-judge scores. Mean $\pm$ standard deviation is the correct statistical framing. Running too few evaluations produces unreliable estimates. Reporting only the best score is cherry-picking and gives a misleading picture of production performance.

Q5. An LLM correctly answers questions about events up to its knowledge cutoff but confidently gives wrong answers about recent events after the cutoff. What is the most dangerous aspect of this behaviour?

- A. The model is slow to answer recent questions.
- B. The model doesn't signal that its knowledge has a cutoff — it produces confident, plausible-sounding but outdated or fabricated answers for post-cutoff events.**
- C. The model's token count increases for recent questions.
- D. The model refuses to answer any recent questions.

Correct Answer: B

Explanation

The danger is not that the model lacks knowledge — it's that it doesn't express uncertainty about what it cannot know. Confident fabrication about recent events (elections, company news, scientific findings) can cause real harm. Systems serving time-sensitive information must use retrieval grounding and prompt the model to express uncertainty on post-cutoff topics.

Q6. What is 'sycophancy' in LLMs, and why is it a problem for reliability?

- A. The model copies text directly from training data, causing copyright issues.
- B. The model agrees with or validates user statements even when they are wrong, prioritising user approval over accuracy.**
- C. The model produces excessively long outputs when given longer inputs.
- D. The model's outputs change based on user demographic information.

Correct Answer: B

Explanation

Sycophancy emerges from RLHF training: human raters often prefer responses that validate their views, training the model to agree rather than correct. This is a reliability problem — users relying on the model for fact-checking or decision support get false validation instead of accurate information.

Q7. A production LLM application has a 95% user satisfaction score but a 15% factual error rate on a gold-standard evaluation set. How should this discrepancy be interpreted?

- A. User satisfaction is the correct metric — the evaluation set must be wrong.
- B. Users cannot reliably detect factual errors in fluent, confident outputs; satisfaction measures perceived quality, not actual accuracy. Both metrics are needed.**
- C. The 15% error rate is acceptable for any AI system.
- D. The evaluation set must not be representative of production inputs.

Correct Answer: B

Explanation

User satisfaction measures perception, not correctness. Fluent, authoritative-sounding errors often pass undetected. High satisfaction + high error rate is a dangerous combination — users trust a system that is wrong. Rigorous evaluation on gold-standard sets is essential alongside satisfaction data.

Q8. You want to test whether an LLM's outputs are factually grounded in the provided context (not hallucinated). Which evaluation approach is most direct?

- A. Ask the model to rate its own confidence.
- B. Use an NLI (natural language inference) model or an LLM judge to check whether each claim in the output is supported by (entailed by) the provided context.**
- C. Compare the output to a keyword list from the context.
- D. Measure the cosine similarity between the output embedding and the context embedding.

Correct Answer: B

Explanation

NLI-based or LLM-judge faithfulness evaluation checks whether each generated claim is entailed by the source context — the direct definition of groundedness. Cosine similarity measures topic similarity, not claim-level entailment. Self-rated confidence is unreliable. Keyword overlap misses paraphrases and misses contradictions.

Q9. An LLM is used for medical advice and occasionally omits a critical piece of information (e.g. a drug interaction warning). Which failure type is this, and how serious is it?

- A. A formatting error — serious but recoverable.
- B. An omission hallucination — potentially life-threatening in a medical context, since missing critical information can be as dangerous as providing false information.**
- C. A temperature artefact — reduce temperature to fix.
- D. A context window issue — provide more input context.

Correct Answer: B

Explanation

Omissions are as dangerous as commissions in high-stakes domains. If a model discusses a medication without mentioning a contraindication, the user receives an incomplete and potentially harmful picture. Medical AI systems require exhaustive coverage testing, not just accuracy testing, with human expert review.

Q10. Which approach best reduces an LLM's tendency to hallucinate in a question-answering system?

- A. Use a larger model.
- B. Use RAG to provide relevant retrieved context, and instruct the model to answer only from the provided context and to say 'I don't know' if the answer is not there.**
- C. Set temperature to 0.
- D. Fine-tune the model on more QA examples.

Correct Answer: B

Explanation

RAG grounds the model's answers in retrieved evidence rather than parametric memory. Explicit instructions to stay within the provided context and acknowledge uncertainty further reduce hallucination. This combination — grounding + uncertainty instruction — is the most effective practical mitigation available.

Q11. A team is comparing two models for production. Model A scores higher on an academic NLP benchmark. Model B scores higher on a custom evaluation using 200 real production examples. Which score should drive the deployment decision?

- A. The academic benchmark — it is more rigorous and standardised.
- B. The custom evaluation — it directly reflects the task and data distribution the model will face in production.**
- C. Both scores equally — average them.
- D. Neither — run a live A/B test and decide based on user metrics.

Correct Answer: B

Explanation

Academic benchmarks measure general capability on standardised tasks. Custom evaluations on production-representative data measure actual task performance. The latter is always more predictive of real-world outcomes. A/B tests add valuable signal but should come after evaluation confirms the model is ready.

Q12. An LLM is asked: 'Is it safe to mix bleach and ammonia?' It answers: 'Many people mix household cleaners.' Why is this response problematic, and what type of failure is it?

- A. The model hallucinated — people don't actually mix cleaners.
- B. The model gave a non-answer that fails to address the safety question — an evasion failure that leaves the user without the critical safety information they need.**
- C. The model's temperature was too high.
- D. The question was outside the model's training distribution.

Correct Answer: B

Explanation

The response is technically not false but is unhelpfully evasive on a clear safety question. This is a common failure mode in over-cautious or misaligned models: avoiding the question rather than providing the correct (and important) information. Bleach + ammonia creates toxic chloramine gas — a model should say so clearly.

Q13. You are building a QA system for a legal firm. The model must never fabricate case citations. Which architecture is most appropriate?

- A. A prompt-only approach: instruct the model not to fabricate citations.
- B. A RAG system where the model is only permitted to cite documents explicitly present in the retrieved context, with a post-generation citation verification step.**
- C. A fine-tuned model trained on verified legal documents.
- D. A model with a high temperature to maximise coverage of possible citations.

Correct Answer: B

Explanation

Instruction alone cannot prevent citation hallucination — the model will still generate plausible-looking but fabricated citations under pressure. RAG constrains citable sources to the retrieved set; post-generation verification confirms each cited document exists in the retrieval index. This is the only architecture that structurally prevents citation fabrication.

Q14. What does a high 'faithfulness' score in a RAG evaluation mean?

- A. The retrieved documents are highly relevant to the query.
- B. The model's generated answer is well-supported by the retrieved context, with no claims that go beyond or contradict the retrieved evidence.**
- C. The model's answer is factually correct according to ground truth.
- D. The retrieval step returns documents with high embedding similarity.

Correct Answer: B

Explanation

Faithfulness (or groundedness) measures whether the generated answer is supported by the retrieved context — not whether the context itself is correct. A model can be faithful to retrieved context that is itself wrong. Faithfulness and factual correctness are distinct dimensions that both need to be evaluated.

Q15. A model produces a correct answer but with low confidence when the correct answer appears in the middle of a long context. What phenomenon explains this?

- A. The model's vocabulary is too small to handle middle-context tokens.
- B. The 'lost in the middle' effect — LLMs tend to underweight information positioned in the middle of long contexts relative to the start and end.**
- C. The model's attention mechanism is limited to the first and last 1,000 tokens.
- D. The context window was exceeded, causing middle tokens to be dropped.

Correct Answer: B

Explanation

Research has demonstrated that transformer models attend most strongly to tokens at the beginning and end of long contexts. Information buried in the middle of a 50k-token context may receive substantially less attention than the same information placed near the start, even if the full context fits in the context window.

SET

5

Multimodal, Deployment & Emerging Capabilities

Set 5 of 5 · 15 Questions · LLM Fundamentals Question Bank

Q1. A multimodal LLM is asked to describe the content of a complex engineering diagram. It produces a fluent description that misses critical details. What is the most likely cause?

- A. The model's text generation is too slow for image inputs.
- B. The model's vision encoder may not be trained on domain-specific technical diagrams, causing it to correctly detect general visual patterns but miss specialised symbols or notation.**
- C. Engineering diagrams exceed the model's image resolution limit.
- D. The model hallucinates more on image inputs than on text inputs.

Correct Answer: B

Explanation

Vision-language models are trained on general image-text pairs. Specialised technical diagrams (circuit schematics, engineering drawings, medical imaging) use domain-specific notation the model may not have seen sufficiently in training. The result is fluent but superficial description missing domain-critical detail.

Q2. Your team is deploying an LLM API. Latency per request spikes from 800ms to 4,000ms when concurrent requests reach 50. Which is the most likely bottleneck?

- A. The model's tokeniser is slow at 50 concurrent requests.
- B. GPU memory bandwidth is saturated — the model is serving requests sequentially instead of batching effectively, causing queuing delay.**
- C. The API's rate limiter is throttling at 50 concurrent requests.
- D. The model's context window fills up with concurrent requests.

Correct Answer: B

Explanation

LLM inference is GPU-bound. At high concurrency, if the serving infrastructure doesn't batch requests efficiently, GPU memory bandwidth becomes the bottleneck and requests queue. Proper batching, efficient KV-cache management, and horizontal scaling are the solutions — not model-level changes.

Q3. What is 'speculative decoding' and what problem does it solve in LLM inference?

- A. A training technique that improves model accuracy by training on speculative future data.
- B. An inference optimisation where a small draft model generates candidate tokens quickly, and the large model verifies them in parallel — reducing wall-clock latency without changing output quality.**
- C. A prompting strategy that asks the model to speculate about possible outcomes.
- D. A quantisation method that reduces model size by speculating on less important weights.

Correct Answer: B

Explanation

Speculative decoding exploits the fact that verifying tokens is cheaper than generating them. The small draft model generates a candidate sequence; the large model checks all tokens in a single forward pass. If tokens are accepted, this is faster than generating them one by one, reducing latency significantly for long outputs.

Q4. An LLM is used to extract structured data from invoices (PDF → JSON). The model works well on English invoices but poorly on invoices in German. The German invoices use the same date and currency formats. What is the most targeted fix?

- A. Increase max_tokens to give the model more room for German outputs.
- B. Add a few German invoice examples to the prompt (few-shot) and verify the extraction field names match German invoice terminology.**
- C. Switch to a larger model with better multilingual capability.
- D. Translate German invoices to English before processing.

Correct Answer: B

Explanation

Few-shot examples in the target language anchor the model to the expected input format and output structure. German invoices may use different field labels (e.g. 'Rechnungsnummer' not 'Invoice Number') that require explicit mapping. Few-shot is the fastest fix; translation adds latency and cost and can introduce errors.

Q5. What is 'quantisation' in the context of LLM deployment, and what trade-off does it introduce?

- A. Reducing the number of layers in a model to decrease inference time, at the cost of accuracy.
- B. Representing model weights in lower precision (e.g. 4-bit instead of 32-bit float), reducing memory footprint and improving inference speed at a small cost to output quality.**
- C. Compressing the model's training data to reduce storage costs.
- D. Restricting the model's vocabulary to improve inference speed.

Correct Answer: B

Explanation

Quantisation reduces the number of bits used to store each weight. A 7B model in float32 requires ~28 GB; in 4-bit, approximately 3.5 GB. This enables deployment on consumer GPUs or edge devices. The trade-off is minor quality degradation on complex tasks — usually acceptable for most production use cases.

Q6. A team builds a 'mixture of experts' (MoE) LLM. For most queries, only 2 of 8 expert networks are active. What is the primary advantage of this architecture?

- A. MoE models are easier to fine-tune than dense models.
- B. MoE models achieve the parameter count and capacity of a large dense model while using the compute of a much smaller one per token, since most experts are idle at any given step.**
- C. MoE models have smaller context windows but better accuracy.
- D. MoE models eliminate hallucinations by routing sensitive queries to specialised experts.

Correct Answer: B

Explanation

In MoE, each forward pass activates only a subset of experts (e.g. 2 of 8). This means the model has the total representational capacity of all experts but the compute cost of only the active ones. It's how models like Mixtral achieve high quality at lower inference cost than equivalently sized dense models.

Q7. You are choosing a deployment strategy for an LLM application serving 10 requests per day for a small internal team vs. 100,000 requests per day for a consumer product. Which principle guides the infrastructure decision?

- A. Always use the largest available model to maximise quality.
- B. Match infrastructure scale to load: API-based managed inference for low volume; dedicated GPU instances or custom serving for high volume where per-token cost compounds significantly.**
- C. Always self-host to maintain data privacy.
- D. Use the same infrastructure for both to reduce operational complexity.

Correct Answer: B

Explanation

At 10 requests/day, a managed API is the correct choice — no infrastructure overhead, pay-per-use. At 100k requests/day, per-token API pricing can exceed the cost of dedicated GPU instances. Infrastructure decisions must be driven by economics and scale, not defaults.

Q8. An LLM with vision capability is used to verify whether a product image matches a product description. It reports a match for an image that clearly doesn't match. What is this failure called and what causes it?

- A. Context window overflow — the image uses too many tokens.
- B. Multimodal sycophancy — the model biases toward confirming the text description because the training signal rewards agreement between text and image captions.**
- C. Tokenisation error — image tokens are misaligned with text tokens.
- D. Quantisation error — lower precision causes vision failure.

Correct Answer: B

Explanation

Vision-language models trained on image-caption pairs learn to find alignments between text and images. This can cause the model to over-confirm presented text descriptions even when the image contradicts them — a form of multimodal sycophancy. Critical verification tasks need adversarial testing with known mismatches.

Q9. What is 'KV cache' (key-value cache) in LLM serving and why does it matter for production?

- A. A database that caches user queries to avoid reprocessing identical inputs.
- B. A memory structure that stores the attention key and value tensors for already-processed tokens, enabling the model to avoid recomputing them for each new token — dramatically reducing inference latency for long contexts.**
- C. A cache that stores model weights in GPU memory for fast loading.
- D. A mechanism that caches the most frequent tokens to speed up tokenisation.

Correct Answer: B

Explanation

Without KV cache, generating each new token would require reprocessing the entire context from scratch. KV cache stores the intermediate attention states for all previous tokens, making each subsequent token generation a fraction of the compute of the first. It's the primary mechanism that makes long-context inference practical.

Q10. An enterprise wants to deploy an LLM that never sends any user data to an external cloud provider. What is the correct deployment pattern?

- A. Use a commercial API with a data processing agreement.
- B. Deploy a self-hosted open-weight model on the company's own infrastructure or private cloud.**
- C. Use a commercial API with data encryption.
- D. Fine-tune the model on anonymised data before using the commercial API.

Correct Answer: B

Explanation

If no data can leave the enterprise perimeter, a self-hosted open-weight model (Llama, Mistral, Falcon, etc.) on private infrastructure is the only option. Commercial APIs — even with encryption and DPAs — transmit data to external servers. Self-hosting requires infrastructure investment but is the only way to guarantee zero external data transmission.

Q11. What is the role of 'system fingerprinting' in detecting when a model provider has silently updated their model in a production environment?

- A. Embedding a unique token in system prompts that the model echoes back.
- B. Running a fixed set of regression test prompts with deterministic settings (temperature = 0) against a golden output set on a scheduled cadence to detect behavioural drift.**
- C. Querying the API's model version endpoint before each request.
- D. Logging all API responses and comparing token distributions weekly.

Correct Answer: B

Explanation

Model version endpoints may not reflect silent updates. Regression tests with temperature = 0 against a golden output set provide a deterministic signal: if outputs change, the model changed. Scheduled regression testing is the production-standard approach to detecting silent model drift from managed API providers.

Q12. A product team wants to add an LLM feature that suggests replies to user emails. Which privacy consideration is most critical before launch?

- A. The LLM may generate replies that are too long.
- B. Email content is highly personal — the team must determine whether email content is sent to a third-party LLM API, whether users consent to this, and how data is retained by the provider.**
- C. The LLM may not understand informal email language.
- D. Email suggestion features require a different model than general LLMs.

Correct Answer: B

Explanation

Email content contains sensitive personal, professional, and potentially legally privileged information. Sending it to a third-party API without user consent and clear data retention policies is a privacy and legal risk. GDPR, CCPA, and other regulations require explicit consent and transparency before processing personal communications.

Q13. What distinguishes 'reasoning models' (e.g. o1, o3) from standard instruction-tuned models in terms of how they generate answers?

- A. Reasoning models are larger and use more parameters.
- B. Reasoning models generate an extended internal chain-of-thought before producing the final answer, spending more tokens on deliberation — trading latency and cost for improved accuracy on complex multi-step problems.**
- C. Reasoning models use retrieval instead of parametric knowledge.
- D. Reasoning models are fine-tuned on more data than instruction-tuned models.

Correct Answer: B

Explanation

Reasoning models (OpenAI o1/o3, DeepSeek-R1) generate long internal reasoning traces before the final answer. This extended 'thinking' process allows them to decompose complex problems, check intermediate steps, and correct errors — significantly improving accuracy on math, coding, and multi-step reasoning at the cost of higher latency and token usage.

Q14. A team is building an LLM-powered code review tool. The model suggests insecure code patterns 5% of the time. What is the most important thing to add to the system before launch?

- A. A faster model to reduce review time.
- B. A human security engineer review step for any code changes the LLM flags as safe, along with static analysis tooling to catch LLM errors in security-critical code.**
- C. A prompt instructing the model to prioritise security.
- D. A larger model with a better code benchmark score.

Correct Answer: B

Explanation

A 5% rate of insecure suggestions is unacceptable without a safety net in security-critical contexts. Human oversight combined with static analysis tools (SAST) provides the error-catching layer the LLM cannot provide itself. Prompting the model to 'prioritise security' reduces but does not eliminate the failure rate.

Q15. Which statement about 'emergent capabilities' in LLMs is most accurate for a practitioner planning deployments?

- A. Emergent capabilities are predictable from model size and can be planned for in advance.
- B. Certain capabilities appear suddenly and unpredictably at scale thresholds, meaning smaller models used in testing may not exhibit behaviours that the production-scale model will — requiring production-scale evaluation.**
- C. Emergent capabilities only apply to multimodal models.
- D. Emergent capabilities are a marketing term with no practical significance.

Correct Answer: B

Explanation

Emergent capabilities (e.g. in-context learning, multi-step arithmetic, chain-of-thought) appear abruptly beyond certain scale thresholds and cannot be reliably predicted by testing smaller models. This means practitioners cannot fully evaluate a 70B model's capabilities by testing a 7B proxy — production-scale evaluation is necessary for capability-sensitive deployments.

menler

Your turning point in the AI era.

END OF ASSESSMENT

*Your turning
point starts
here.*